Übungsblatt 3: Speicherzugriff und Kontrollfluss

Papieraufgaben, Rechenwege und Venus-Vergleich

Musterlösung — Zahlen und Zustandsfolgen sind Modellvorhersagen. Die eigenen Venus-Beobachtungen werden getrennt protokolliert.

Ziel und Arbeitsweise

Sie berechnen Speicheradressen, verfolgen veränderte Register und Byte und begründen Sprungpfade sowie Schleifengrenzen. Die Inhalte von Vorlesung 1 und 2 bilden das Vorwissen; die neuen Befehle werden in Vorlesung 3 erklärt. Eine höhere Programmiersprache wird nicht vorausgesetzt.

Bearbeiten Sie Aufgaben 1–6 zunächst ohne Simulator. Die **60 Punkte** dienen der Selbstkontrolle und sind keine Prüfungsbewertung. Erst Aufgabe 7 verlangt den Vergleich mit der Kursdistribution von Venus. Die optionale Vertiefung ist zusätzliche Reserve.

Aufgabe	Schwerpunkt	Minuten	Punkte
1	Lesen, rechnen, speichern	9	10
2	Effektive Adresse und Ausrichtung	8	8
3	Teilzugriffe und Bytezustände	9	10
4	Store-Format codieren/decodieren	9	10
5	Entscheidungen und Sprungpfade	10	12
6	Schleifen-Trace und leerer Fall	9	10
7	Array-Summe in Venus	30	—

Planungsannahme für vier Schulstunden: 76 Minuten Vorlesung einschließlich Rückfragen, 54 Minuten Papierarbeit, 30 Minuten Venus, zehn Minuten Quiz und zehn Minuten Auswertung. Bei höherem Unterstützungsbedarf kann Aufgabe 4 nachbesprochen werden; die Vertiefung liegt außerhalb dieser 180 Minuten.

Gemeinsame Annahmen

RV32I ist die RISC-V-Integerbasis mit 32-Bit-Registern. Wir verwenden keine komprimierte Erweiterung C: Jeder native Befehl belegt vier Byte. Adressen zählen Byte. Daten werden Little Endian abgelegt, das niederwertige Byte zuerst. Registerarithmetik speichert die unteren 32 Bit; `x0` liefert null und verwirft Schreibversuche. In Speicheraufgaben werden nur ausdrücklich angegebene Regionen und natürlich ausgerichtete Zugriffe benutzt.

Aliasnamen: `t0=x5`, `t1=x6`, `t2=x7`, `t3=x28`, `t4=x29`, `t5=x30`, `t6=x31`, `s0=x8`, `s1=x9`, `a0=x10`, `a1=x11`. `PC` bedeutet Program Counter, Programmzähler. Nicht genannte Register- oder Speicherinhalte sind unbekannt; jede Aufgabe hat ihren eigenen Anfangszustand.

Aufgabe 1: Lesen, rechnen, speichern

10 Punkte · 9 Minuten · Papier

Eine gültige, beschreibbare Region ab `0x1000` enthält die fünf Wörter $A = [10, 20, 30, 40, 50]$. `s1=0x1000` bleibt zunächst die Basis. Es werden nacheinander ausgeführt:

```
lw   t0, 8(s1)
addi t1, t0, 12
sw   t1, 12(s1)
```

a) Notieren Sie nach jeder Zeile die effektive Adresse, soweit ein Speicherzugriff stattfindet, den neuen Registerinhalt und die veränderten Speicherbyte. Ein Strich bedeutet „keine Änderung“ beziehungsweise „kein Zugriff“ (je 2 Punkte).

b) Schreiben Sie das vollständige Array nach Zeile 3 auf. Welches Quellregister liefert beim Store die Daten, welches die Adresse? (2 Punkte).

c) Welche vier Byte liegen danach an den Adressen `0x100C` bis `0x100F`? Geben Sie außerdem den Endwert von `s1` an (2 Punkte).

Musterlösung

Zeile	effektive Adresse	Registeränderung	Speicheränderung
1	`0x1008`	`t0=30=0x0000001E`	keine
2	kein Zugriff	`t1=42=0x0000002A`	keine
3	`0x100C`	keine	`0x100C: 2A`, folgende drei Byte `00`

b) Das Array lautet $[10, 20, 30, 42, 50]$. `t1` liefert das Datenwort, `s1` die Basis für die Adressrechnung plus zwölf. Links im Store steht die Quelle, kein Zielregister.

c) Bytefolge: `2A 00 00 00`. `s1` bleibt `0x1000`; ein Offset verschiebt die Basis nicht dauerhaft. `t0=30` und `t1=42` bleiben beim Store ebenfalls erhalten. Nur die vier adressierten Speicherbyte werden überschrieben.

Aufgabe 2: Ausrichtung ist eine Adressfrage

8 Punkte · 8 Minuten · Papier

Nur die Byteadressen `0x1000` bis einschließlich `0x1013` seien hier zugreifbar. Bestimmen Sie die effektive Adresse und entscheiden Sie unabhängig über natürliche Ausrichtung und gültigen gesamten Bytebereich (je Zeile 1 Punkt).

Die Breiten sind Wort = 4 Byte, Halbwort = 2 Byte, Byte = 1 Byte.

b) Warum genügt ein gerader Offset nicht, um einen Halbwortzugriff auszurichten? Muss ein unausgerichteter Zugriff auf jeder RISC-V-Implementierung zwingend mit einer Ausnahme abbrechen? (2 Punkte).

Musterlösung

Basis	Offset	Breite	Adresse	ausgerichtet?	Bereich gültig?
`0x1001`	+3	Wort	`0x1004`	ja	ja
`0x1001`	+2	Halbwort	`0x1003`	nein	ja
`0x1002`	+2	Wort	`0x1004`	ja	ja
`0x1003`	−3	Wort	`0x1000`	ja	ja
`0x1003`	0	Byte	`0x1003`	ja	ja
`0x1014`	0	Wort	`0x1014`	ja	nein

Ein Wort muss bei einer durch vier, ein Halbwort bei einer durch zwei teilbaren effektiven Adresse starten. Die gültige Region muss außerdem alle betroffenen Byte enthalten; eine ausgerichtete Adresse allein genügt nicht.

b) Entscheidend ist Basis plus Offset. Eine ungerade Basis plus gerader Offset bleibt ungerade, wie in Zeile 2. Die Behandlung unausgerichteter Zugriffe hängt von der Umgebung ab; ein universeller zwingender Abbruch wird nicht behauptet. Das Kursmodell verwendet natürliche Ausrichtung. Für eine unzugängliche Region darf ebenso kein normaler erfolgreicher Load vorausgesetzt werden.

Aufgabe 3: Teilzugriffe vollständig verfolgen

10 Punkte · 9 Minuten · Papier

Es gilt `s1=0x2000`. Die gültigen Byte sind anfangs:

Adresse	`0x2000`	`0x2001`	`0x2002`	`0x2003`
Inhalt	`80`	`FF`	`00`	`80`

a) Diese vier Loads finden vor jeder Speicheränderung statt. Bestimmen Sie jeweils das achtstellige Registerwort und den Zahlenwert entsprechend der Erweiterungsart (je 1 Punkt).

b) Anschließend ist `t4=0x11223344` gegeben. Es werden `sb t4, 1(s1)` und danach `sh t4, 2(s1)` ausgeführt. Geben Sie die vier Speicherbyte nach jedem Store und das abschließende 32-Bit-Wort ab `0x2000` an (4 Punkte).

c) Bleibt `t4` unverändert? Warum ist für diese Wirkung kein eigenes `sbu` nötig? (2 Punkte).

Musterlösung

Load	Wort	Zahlenwert
`lb t0, 0(s1)`	`0xFFFFFF80`	−128
`lbu t1, 0(s1)`	`0x00000080`	128
`lh t2, 2(s1)`	`0xFFFF8000`	−32768
`lhu t3, 0(s1)`	`0x0000FF80`	65408

b) Nach `sb`: `80 44 00 80`. Nach `sh`: `80 44 44 33`. Das zusammengesetzte Little-Endian-Wort ist `0x33444480`. `sb` schreibt nur `44`, `sh` die unteren sechzehn Bits `3344` als Bytefolge `44 33`.

c) `t4` bleibt `0x11223344`. Der Store kürzt auf die zu schreibenden unteren Bits; es gibt keine oberen Zielbits zu erweitern. Die signed oder unsigned Herkunft des Quellmusters ändert diese Bits nicht, daher ist `sbu` kein eigener nativer Befehl der Basis.

Aufgabe 4: S-Format von Hand

10 Punkte · 9 Minuten · Papier

Referenz: Bits 31:25 = `imm[11:5]`, 24:20 = `rs2`, 19:15 = `rs1`, 14:12 = `funct3`, 11:7 = `imm[4:0]`, 6:0 = `opcode`. Für `sw`: funct3 `010`, opcode `0100011`. Registerfelder haben fünf Bit; `t0=x5`, `t1=x6`, `s1=x9`.

a) Codieren Sie `sw t1, -8(s1)`. Zeigen Sie das zwölf Bit breite Offsetmuster, seine beiden Teile, alle sechs Felder und das Hexwort (6 Punkte).

b) Decodieren Sie `0x0054A623`. Rekonstruieren Sie den Offset und schreiben Sie den Befehl mit Aliasnamen. Welches Registerfeld fehlt diesem Format gegenüber dem Registerformat? (4 Punkte).

Musterlösung

a) $2^{12} - 8 = 4088$: `0xFF8` beziehungsweise `111111111000`. Oben stehen `1111111`, unten `11000`. `rs2=x6` wird `00110`, `rs1=x9` wird `01001`. Das Wort lautet:

```
1111111 00110 01001 010 11000 0100011
1111 1110 0110 0100 1010 1100 0010 0011
   F    E    6    4    A    C    2    3
```

Ergebnis: `0xFE64AC23`. Die Rückprüfung verbindet die Teile zu `0xFF8` und liest $4088 - 4096 = -8$.

b) Zerlegung:

```
0000000 00101 01001 010 01100 0100011
```

Store-Familie und funct3 ergeben `sw`. `rs2=5=t0`, `rs1=9=s1`, Immediate `000000001100` = zwölf. Also `sw t0, 12(s1)`. Es gibt kein `rd`, weil der Zielort im Speicher liegt. Die Bits 11:7 tragen den unteren Offsetteil und sind keine Zielregisternummer.

Aufgabe 5: Welche Instruktionen laufen wirklich?

12 Punkte · 10 Minuten · Papier

Der erste Befehl liegt bei `0x1000`. Alle gezeigten Befehle sind **nativ**. `jal x0, EndIf` ist hier die feste Entwicklung von `j EndIf` und verwirft den Linkwert durch `x0`.

```
    bne  s0, s1, ElseBlock
    addi t0, x0, 1
    jal  x0, EndIf
ElseBlock:
    addi t0, x0, 2
EndIf:
    addi t1, x0, 9
```

a) Geben Sie die Adressen der fünf Instruktionen und der beiden Labels an (3 Punkte).

b) Notieren Sie für `s0=s1=7` und für `s0=7, s1=8` die ausgeführten Befehlsadressen in Reihenfolge, die Adresse nach dem letzten Befehl und die Endwerte `t0` und `t1` (4 Punkte).

c) Welche signed Byteabstände werden beim Branch und beim Jump zum jeweiligen Ziel verwendet? Der Abstand zählt ab der Sprunginstruktion (2 Punkte).

d) Ergänzen Sie die genommene/nicht genommene Entscheidung für `blt` und `bltu` mit Quellen `0xFFFFFFFF` und `1` in dieser Reihenfolge (2 Punkte).

e) Eine Variante lässt den Jump zwischen den Blöcken weg. Welcher Fehler entsteht für gleiche Eingaben? Eine neue Adressrechnung ist nicht verlangt (1 Punkt).

Musterlösung

a) Befehle: `0x1000`, `0x1004`, `0x1008`, `0x100C`, `0x1010`. `ElseBlock=0x100C` und `EndIf=0x1010`. Labels erzeugen keine zusätzlichen Wörter.

b) Gleich: `1000 → 1004 → 1008 → 1010`, danach `1014`. Endwerte `t0=1`, `t1=9`. Ungleich: `1000 → 100C → 1010`, danach `1014`. Endwerte `t0=2`, `t1=9`. Alle Adressen sind hexadezimal. Die Zeile an `100C` wird nur im Ungleichheitsfall ausgeführt.

c) Branch: `0x100C−0x1000` = zwölf Byte. Jump: `0x1010−0x1008` = acht Byte. Nicht vom jeweiligen PC plus vier rechnen.

d) `blt` wird genommen, weil signed minus eins kleiner als eins ist. `bltu` wird nicht genommen, weil unsigned 4294967295 größer als eins ist.

e) Der Gleichheitsweg schreibt erst eins, läuft aber anschließend in Else und überschreibt das Ergebnis mit zwei. Ein Label begrenzt keinen Block automatisch.

Aufgabe 6: Eine Schleife einschließlich Endtest

10 Punkte · 9 Minuten · Papier

Ein gültiges Array mit genau drei Wörtern $A = [4, -1, 2]$ beginnt bei $B = \texttt{0x3000}$. Anfang: `t0=0`, `t1=3`, `s0=0`, `s1=B`. `t2` ist zunächst unbekannt.

```
Loop:
    bge  t0, t1, Done
    lw   t2, 0(s1)
    add  s0, s0, t2
    addi s1, s1, 4
    addi t0, t0, 1
    j    Loop
Done:
```

a) Füllen Sie alle Zustände **vor** `bge` aus, einschließlich des Tests, der beendet (4 Punkte).

b) Wie viele Loads und Tests werden ausgeführt? Welches Wort und welchen signed Wert hat `t2` am Ende? (2 Punkte).

c) Setzen Sie nur die Anzahl auf `t1=0` und beginnen Sie neu. Wie lauten Summe, Pointer und Anzahl der Loads? Muss `t2` dafür vorher null sein? (2 Punkte).

d) Warum sind ein Pointerupdate um eins oder ein Limit vier für dieses Array unpassend? Unterscheiden Sie Ausrichtung und Objektgrenze (2 Punkte).

Musterlösung

Test	`t0`	`s1`	`s0`	genommen?
1	0	`0x3000`	0	nein
2	1	`0x3004`	4	nein
3	2	`0x3008`	3	nein
4	3	`0x300C`	5	ja

b) Drei Loads, vier Tests. Der letzte Load war das Wort zwei, also `t2=0x00000002`. Das negative Element davor wurde als `0xFFFFFFFF` geladen; $4 + (-1) = 3$ folgt der 32-Bit-Arithmetik.

c) Sofortiger Endtest: Summe null, Pointer `0x3000`, null Loads. `t2` wird nicht gelesen und darf unbekannt bleiben. Die Initialisierung von Summe und Zähler ist dagegen notwendig.

d) Pointer plus eins würde den zweiten Wortzugriff an `0x3001` legen und die natürliche Ausrichtung verfehlen. Limit vier würde zusätzlich an `0x300C` laden: ausgerichtet, aber hinter den drei angelegten Wörtern. Das Programm kennt die Arraylänge nur durch die richtige Vorbedingung und Grenze.

Aufgabe 7: Array-Summe in Venus

30 Minuten · Rechner und Papierprotokoll

Implementieren Sie die Summe der ersten N Werte aus `.word 10, 20, 30, 40, 50`. Vorbedingung: $0 \leq N \leq 5$. Die Eingabedaten bleiben unverändert.

1. Legen Sie die Daten unter `myArray:` im Datenabschnitt an. Nutzen Sie einen Codeabschnitt mit `.globl main` und Startlabel `main:`.
2. Laden Sie die Basis mit `la s1, myArray`. Initialisieren Sie `t0=0`, `t1=N` und `s0=0`.
3. Verwenden Sie das Schleifenmuster aus Aufgabe 6. Der Test liegt vor `lw`, der Pointer wächst um vier Byte, der Zähler um eins.
4. Am Label `Ergebnis_pruefen:` vor der Ausgabe prüfen Sie `s0`, `t0` und `s1`. Notieren Sie vorher die erwarteten Werte.
5. Ausgabe und Ende in Kurs-Venus: `mv a1, s0`, `li a0, 1`, `ecall`, danach `li a0, 10` und `ecall`. Die Dienstnummer ist kein Summenwert.
6. Prüfen Sie $N = 5$, $N = 0$, $N = 1$ und danach den Datensatz $[-3, 8, -10, 1]$ mit $N = 4$. Nach jeder Änderung neu assemblieren und starten. Protokollieren Sie Ergebnis, Pointerabstand zur tatsächlichen Basis und Load-Anzahl.

Musterlösung

Die vollständige Quelle steht in vl3_array_summe.s. Daten und Anfang stehen vor `Loop`; der Rumpf entspricht Aufgabe 6. Vor der Ausgabe gilt:

Fall	`s0` / Druckzahl	`t0`	Pointer	Loads / Tests
$N = 5$	150	5	$B + 20$	5 / 6
$N = 0$	0	0	B	0 / 1
$N = 1$	10	1	$B + 4$	1 / 2
gemischte Werte, $N = 4$	−4 = `0xFFFFFFFC`	4	$B + 16$	4 / 5

Im gemischten Fall sind die Teilsummen $-3, 5, -5, -4$. Der Prüfpunkt liegt vor `mv a1, s0`. Danach ist `a0` zuerst Dienst 1 zum Drucken, schließlich Dienst 10 zum Beenden. Die Tabelle ist eine Modellvorhersage; das tatsächliche Venus-Protokoll ist getrennt einzutragen. Das Beispiel setzt keine allgemeine Erhaltung aller Register durch beliebige Dienste voraus.

Optionale Vertiefung: Teilzugriffe vergleichen

Zusätzliche Reserve · nicht Teil der 84 Übungsminuten

Öffnen Sie vl3_teilzugriffe.s. Der Prüfpunkt liegt vor dem Abschlussdienst. Sagen Sie zunächst die Werte von `t0` bis `t6` und die beiden Wörter bei `target` voraus. Vergleichen Sie dann mit den Registern und der Speicheransicht. Klären Sie, ob die Ansicht einzelne Byte oder zusammengesetzte Wörter zeigt.

Öffnen Sie anschließend vl3_if_else.s. Prüfen Sie gleiche und ungleiche Eingaben. Notieren Sie die tatsächlich ausgeführten Instruktionen, nicht nur die letzte Ergebniszahl. Erst nach der Vorhersage können Sie in einer Kopie den Jump entfernen und die veränderte Folge erklären.

Musterlösung

Teilzugriffe: `t0=0xFFFFFF80`, `t1=0x00000080`, `t2=0xFFFFFFFF`, `t3=0x000000FF`, `t4=0x0000007F`, `t5=0xFFFF8000` und `t6=0x00008000`. Die Bytequellen bleiben unverändert.

Bei `target` lauten die Wörter danach `0xAAAAAA44` und `0xBBBB3344`. `sb` verändert das erste Byte des ersten Worts, `sh` die ersten zwei Byte des zweiten. Die Registerquelle `a2=0x11223344` bleibt vor dem Exit unverändert.

Entscheidung: Bei `s0=s1=7` steht `t0=1` am Prüfpunkt. Für `s0=7, s1=8` steht dort zwei. Ohne den Jump führt der Gleichheitsweg beide Initialisierungen aus und endet ebenfalls bei zwei. Diese Angaben sind Sollwerte des Modells, keine vorausgefüllte Werkzeugbeobachtung.